

NLP Assignment 1 Q1 Report

Muhammad Umer Siddiqui
ms08702@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abdullah Shaikh
as09245@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Arqish
az09714@st.habib.pk
Dhanani School of Science and
Engineering, Habib University
Karachi, Pakistan

Abstract

This report discusses the solution of Question 1 in Assignment 1 for the Natural Language Processing course offered in Fall 2026 at Habib University. It analyzes design choices and the effect of varying hyperparameters of WordPiece and BPE tokenizers on a small multilingual text corpus containing Urdu, Chinese, and English.

Keywords

Natural Language Processing, Tokenization, BPE, WordPiece, Multilingual NLP, Subword Tokenization

1 Introduction

The question provided a small multilingual corpus in English, Urdu, and Chinese. It tasked us with implementing the WordPiece and Byte Pair Encoding tokenizers from scratch and then experimenting with different hyperparameters for the two algorithms on the given dataset. Our implementation had one hyperparameter, namely vocabulary size. The algorithms were implemented in `q1.py`, and the resulting vocabularies were stored in `bpe_vocab.txt` and `wordpiece_vocab.txt`.

2 Preprocessing

The text first underwent Unicode Normalization Form C, ensuring that all visible characters were represented by the same Unicode sequence. This ensured that our algorithms did not treat any visually identical characters differently. The text was then converted to lowercase, which was a change that would have only impacted the English text in our corpus. This ensured that capitalization of individual letters in a word did not lead to the algorithm classifying them as new words. Following this, punctuation marks in all three languages were separated from normal words and stored. Finally, all whitespace was normalized to a single space. All of these steps impacted all three languages, except the conversion to lowercase, which impacted only English.

3 Tokenization Methods

A parent class `Tokenizer` was built for both tokenizers to inherit from in order to avoid repeating any methods. The class dealt with taking the initial input, preprocessing the corpus, and writing the output vocabulary to a file. The remaining methods were implemented in the child classes.

3.1 WordPiece Algorithm

The `WordPieceTokenizer` class had two methods. The first method, `tokenize`, took the preprocessed text corpus, split each word into individual characters, added `##` to each character that was not at

the start of a word, and stored the resulting sequences in a two-dimensional list called `tokens`, which the function returned.

The second method, `vocabulary_builder`, first initialized the vocabulary using all the tokens produced by the `tokenize` method. It then repeatedly calculated the frequency of each individual token and the frequency of every adjacent token pair. Instead of selecting the most frequent pair directly, WordPiece calculated a score for each pair using the frequency of the pair divided by the product of the frequencies of its two individual tokens. The pair with the highest score was selected for merging.

The selected pair was then combined into a new token, with the `##` marker removed from the second token during the merge. All occurrences of the selected adjacent pair in the token sequences were replaced by the new token. The new token was added to the vocabulary, and this process was repeated until the specified vocabulary size was reached or no further pairs were available.

3.2 Byte Pair Encoding Algorithm

The `BPETokenizer` class also had two methods. The `tokenize` method took the preprocessed corpus, split it into words, and represented each word as a list of its individual characters. Unlike WordPiece, it did not add `##` markers to characters occurring after the first character of a word.

The second method, `vocabulary_builder`, initialized the vocabulary using all unique characters in the corpus. It then repeatedly calculated the frequency of every pair of adjacent tokens. Unlike WordPiece, BPE selected the pair with the highest raw frequency.

After selecting the most frequent pair, the two tokens were concatenated to form a new token. All occurrences of that pair in the token sequences were then replaced with the new token. The new token was added to the vocabulary, and the process was repeated until the specified vocabulary size was reached or no further pairs were available.

The main difference between the two approaches was therefore the criterion used to select pairs for merging. BPE selected the most frequently occurring adjacent pair, whereas WordPiece used a normalized score based on both pair frequency and the frequencies of the individual tokens.

4 Experiments and Results

The initial vocabulary size was randomly chosen as 250. This was a particularly terrible choice for WordPiece as it did not learn a single token longer than one character. BPE did noticeably better, learning words like `word` and `tokeniz`.

Increasing the vocabulary size to 300 did not have any observable impact on WordPiece, but BPE learned many more longer strings like `tokenization` and `process` along with suffixes like `ing`.

Increasing the vocabulary size to 350 had approximately the same

impact as increasing it from 250 to 300.

Increasing the vocabulary size to 450 showed noticeable improvement for WordPiece, especially in the Urdu language, as it started to learn proper words. The BPE tokenizer also continued to improve. Increasing vocabulary size to 500 did not show any significant change in either.

Increasing the vocabulary size for WordPiece to 600 led to it finally learning a substantial number of English subword strings. BPE was

learning a substantial number of full words at this stage.

Finally, increasing the vocabulary size to 700 provided decent results for WordPiece, and further improved what BPE learned as well. Further increases to 800 and 900 confirmed this trend for WordPiece.

Based on the observed tokenization results, a vocabulary size of 700 provided a reasonable balance between vocabulary size and the quality of the learned subword tokens for this dataset.